Throws

Nothing.

D.5.2. std::recursive_mutex class

The `std::recursive_mutex` class provides a basic mutual exclusion and synchronization facility for threads that can be used to protect shared data. Prior to accessing the data protected by the mutex, the mutex must be *locked* by calling `lock()` or `try_lock()`. Only one thread may hold the lock at a time, so if another thread also tries to lock the `recursive_mutex`, it will fail (`try_lock`) or block (`lock`) as appropriate. Once a thread is done accessing the shared data, it then must call `unlock()` to release the lock and allow other threads to acquire it.

This mutex is *recursive* so a thread that holds a lock on a particular `std::recursive_mutex` instance may make further calls `lock()` or `try_lock()` to increase the lock count. The mutex can't be locked by another thread until the thread that acquired the locks has called `unlock` once for each successful call to `lock()` or `try_lock()`. `std::recursive_mutex` meets the `Lockable` requirements.

Class definition

```
class recursive_timed_mutex
{
public:
    recursive_timed_mutex(recursive_timed_mutex const&)=delete;
    recursive_timed_mutex& operator=(recursive_timed_mutex
const&)=delete;

    recursive_timed_mutex();
     ~recursive_timed_mutex();

    void lock();
    void unlock();
    bool try_lock() noexcept;
};
```

Std::Recursive_Mutex Default Constructor